

# Asynchronous JavaScript

Def:- JavaScript normally runs synchronously meaning one line executes after another.

But some tasks (like fetching data from a server) take time to prevent the page from "freezing", JS uses asynchronous programming - it lets the rest of the code run while waiting for that slow task to finish.

Examples of sync tasks -

- fetching data from an API
- Reading lines (in Node.js)
- Timer functions (set Timeout)

## Promises:-

def:- A promise is an object that represents a task that may complete in the future either successfully or with an error.

(\*) States of promises.

1. Pending - waiting for the task to complete.
2. Resolved (fulfilled) → Task succeeded
3. Rejected - Task failed.

### (\*) Syntax -

```

let promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("task completed");
  }
  else {
    reject("something went wrong");
  }
});

```

### (\*) Using .then() & .catch()

```

=> promise.then((result) => {
  console.log(result);
});

```

```

=> promise.catch((error) => {
  console.log(error);
});

```

Example (Real Use case):

```
function getData () {  
  return new Promise ((resolve) => {  
    setTimeout ( () => {  
      resolve ("Data recieved!"),  
      2000 } )  
    } )  
};
```

getData.then ( (message) => {

```
  console.log ( message ) } ) ;  
// "Data Recieved!" (After 2 seconds)  
  "waits for 2 seconds"
```

## Async / Await -

Def - `async` and `await` are keywords introduced in ES8. They make asynchronous code look and behave like synchronous code, making it easier to read & write.

### (\*) Syntax

```
async function fetchData() {
```

```
  let promise = new Promise ( resolve => {
```

```
    setTimeout ( () => resolve ("Data loaded!"), 2000 );
```

```
  } );
```

```
  let result = await promise;
```

```
  console.log (result);
```

```
}
```

```
fetchData ();
```

// → Output after 2 seconds → Data loaded!

## Key points -

- always use await inside an async function.
- await pauses the function until the promise resolves.
- error can be handled with try...catch.

## Example

async function getUser() {

try {

let res = await fetch("https://api.example.com/user");

let data = await res.json();  
console.log(data);

} catch (error) {  
console.log("Error", error);

}  
};

## Fetch API (to call Backend):

`fetch()` is a built-in JS function to make HTTP requests to servers or APIs.

It returns a Promise, which resolves to a Response object.

### \* Get (GET Request Ex) -

```
fetch ("https://jsonplaceholder.typicode.com/  
      users")
```

```
• then ( res => res.json())
```

```
// convert response to JSON
```

```
• then ( data => console.log (data))
```

```
• catch ( err => console.log ("Error", err));
```

### \* POST Request Example

```
fetch ("https://jsonplaceholder.typicode.com/posts",  
      {  
        method: "POST"  
        headers: {  
          "content type": "application/json",  
        },  
      })
```

```
body : JSON.stringify ({
```

```
  title : "Hello Tee",  
  body : "Learning async JS",  
  user.id : 101
```

```
})
```

```
});
```

- then (res  $\Rightarrow$  res.json())
- then (data  $\Rightarrow$  console.log(data)).
- ~~catch~~ (error  $\Rightarrow$  console.log("error", error));

## Using Fetch with Async / Await.

```
async function createPost() {
```

```
  let res = await  
  fetch ("https://jsonplaceholder.typicode.com/  
         posts", {  
    method : "POST",  
    headers : { "Content-Type": "application/json"},  
    body : JSON.stringify ({ title : "Async  
                          Await", body : "MERN Rocks" })  
  });
```

```
};
```

```
let data = await res.json();  
console.log(data);
```

```
} createPost();
```